

Chapter 22 — Deployment & Operations

22.1 Introduction

Designing a decentralized protocol is only part of the challenge.

Operating it reliably, securely, and efficiently is equally important.

OpenFiat is intended to operate as a continuously available global marketplace.

Merchants in one country should be able to trade with confidence regardless of whether AllenHark's infrastructure is online.

Achieving this requires a network of independent infrastructure providers operating nodes across multiple regions, organizations, cloud providers, and network operators.

This chapter describes how OpenFiat is deployed, operated, monitored, upgraded, and maintained in production.

22.2 Operational Philosophy

OpenFiat follows several operational principles.

No Single Point of Failure

The protocol should continue functioning despite the failure of individual nodes, infrastructure providers, or organizations.

Open Participation

Anyone meeting the protocol requirements may operate infrastructure.

Horizontal Scalability

Capacity should increase naturally as more nodes join the network.

Deterministic Recovery

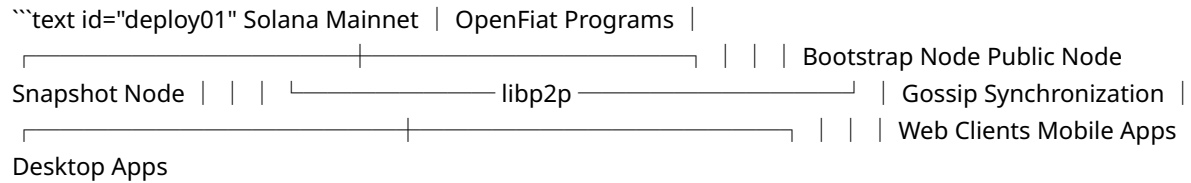
Infrastructure should recover to a consistent state after failures.

Observable Infrastructure

Operators should be able to monitor node health using standardized metrics.

22.3 Production Architecture

A typical production deployment consists of several independent components.



Each component may be operated by different organizations.

No infrastructure provider has authority over protocol operation.

— — —

22.4 Bootstrap Infrastructure

Every new node requires an initial point of contact.

OpenFiat therefore maintains a small number of well-known bootstrap endpoints.

Examples include:

- entry01.openfiat.org
- entry02.openfiat.org
- openfiat.allenhark.com

Bootstrap nodes have only one responsibility:

Introduce newly joined nodes to the existing network.

They do not:

- Validate transactions.
- Store exclusive marketplace data.
- Control governance.
- Approve participants.
- Route protocol decisions.

Once peer discovery completes, nodes communicate directly with one another.

22.5 Official Bootstrap Phase

During the initial network launch, AllenHark will operate core infrastructure including:

- Bootstrap nodes.
- Snapshot servers.
- Public gateway endpoints.
- Public monitoring dashboards.
- Reference APIs.

These services accelerate network adoption.

As community infrastructure grows, the protocol becomes progressively independent of AllenHark.

22.6 Node Deployment

Deploying an OpenFiat node should require minimal configuration.

Typical deployment steps include:

```text id="deploy02"

Install OpenFiat Node

↓

Configure Wallet

↓

Configure Network

↓

Connect Bootstrap Nodes

↓

Download Snapshot

↓

Replay Recent Gossip

↓

Begin Serving Network

The official reference node automatically initializes its local RocksDB database and synchronizes marketplace state.

---

## 22.7 Containerization

The reference implementation is designed to support containerized deployments.

Examples include:

- Docker
- Docker Compose
- Kubernetes
- Nomad
- Podman

Containerization simplifies:

- Upgrades.
- Scaling.
- Monitoring.
- Backup.
- Disaster recovery.

Operators remain free to deploy nodes directly on bare-metal systems if preferred.

---

## 22.8 Hardware Recommendations

Minimum hardware recommendations:

- 4 CPU cores
- 16 GB RAM
- 250 GB NVMe SSD
- Stable broadband connection

Recommended production hardware:

- 8–16 CPU cores
- 32 GB RAM or more
- 1 TB NVMe SSD
- High-bandwidth network connection
- UPS power protection
- Redundant internet connectivity

OpenFiat nodes are intentionally designed to operate on commodity hardware.

Unlike Solana validators, specialized servers are not required.

---

## 22.9 Geographic Distribution

A decentralized marketplace benefits from global infrastructure.

Operators are encouraged to deploy nodes across multiple regions.

Examples include:

- North America
- South America
- Europe
- Africa
- Asia
- Oceania

Geographic diversity improves:

- Availability.
- Fault tolerance.
- Latency.
- Disaster resilience.

No region should become critical to protocol operation.

---

## 22.10 Monitoring

Every node should expose operational metrics.

Examples include:

### Infrastructure

- CPU utilization.
- Memory usage.
- Disk usage.
- Network throughput.

### Protocol

- Connected peers.
- Gossip propagation rate.
- Active advertisements.
- Active trade sessions.
- Snapshot age.
- Synchronization status.

### Service

- API request volume.
- Notification processing.
- Risk intelligence updates.
- Oracle updates.

Monitoring enables operators to detect issues before they affect users.

---

## 22.11 Logging

The reference node produces structured logs.

Examples include:

- Startup events.
- Peer connections.
- Snapshot downloads.
- Gossip synchronization.
- Trade events.
- Infrastructure warnings.
- Upgrade events.
- Error conditions.

Logs should support both human troubleshooting and automated analysis.

---

## 22.12 Health Checks

Every node exposes standardized health endpoints.

Typical health information includes:

- Synchronization status.
- Connected peer count.
- Current protocol version.
- RocksDB integrity.
- Snapshot freshness.
- Gossip status.
- Available services.

Applications may use health information when selecting preferred nodes.

---

## 22.13 Automatic Recovery

Nodes should recover automatically whenever possible.

Examples include:

Internet interruption

↓

Reconnect peers

Node restart

↓

Recover RocksDB

Snapshot corruption

↓

Download latest snapshot

Bootstrap unavailable

↓

Use alternative bootstrap node

Manual intervention should rarely be required.

---

## 22.14 Rolling Upgrades

The protocol supports rolling infrastructure upgrades.

Operators may upgrade nodes individually without requiring simultaneous network-wide downtime.

Upgrade process:

```text id="upgrade01" Download Release

↓

Verify Signature

↓

Graceful Shutdown

↓

Upgrade Software

↓

Restart Node

↓

Replay Missed Gossip

↓

Resume Operation ```

This allows the network to evolve continuously.

22.15 Version Compatibility

Nodes advertise their supported protocol version during peer discovery.

Compatibility policies include:

Patch versions

Fully compatible.

Minor versions

Generally compatible.

Major versions

May require coordinated governance-approved upgrades.

Deprecation schedules are announced well before incompatible versions are retired.

22.16 Backup Strategy

Although marketplace state is replicated throughout the network, operators should maintain local backups.

Recommended backups include:

- Configuration files.
- Node identity.
- Wallet keys.
- RocksDB snapshots.
- Monitoring configuration.

Authoritative financial state remains secured on Solana.

22.17 Disaster Recovery

Infrastructure failures are expected.

The protocol is designed to recover quickly.

Examples include:

Hardware failure

↓

Deploy replacement server

↓

Restore configuration

↓

Download snapshot

↓

Replay gossip

↓

Resume participation

No centralized recovery procedure is required.

22.18 Network Scaling

OpenFiat scales horizontally.

As marketplace activity grows:

- Additional nodes join.
- More snapshot providers appear.
- Additional notification gateways operate.
- More oracle providers publish data.
- More Risk Intelligence Providers participate.

Capacity therefore grows alongside demand.

22.19 Operational Security

Operators should follow standard infrastructure security practices.

Examples include:

- Full disk encryption.
- Firewall configuration.
- Operating system updates.
- SSH key authentication.
- Hardware security modules where appropriate.
- Secure backup procedures.
- Network segmentation.
- Least privilege access.

OpenFiat provides protocol security.

Operators remain responsible for securing their own infrastructure.

22.20 Service Level Expectations

While the protocol does not mandate uptime guarantees, infrastructure providers seeking high reputation should target:

- High availability.
- Low latency.
- Rapid synchronization.
- Reliable storage.
- Consistent software updates.

These operational characteristics contribute directly to marketplace quality and provider rewards.

22.21 AllenHark Operations

During the bootstrap period, AllenHark intends to operate:

- Multiple bootstrap nodes.
- Public snapshots.
- Public Notification Gateways.
- Public Risk Intelligence integration.
- Reference web interface.
- Reference APIs.

These services exist to accelerate ecosystem growth rather than centralize protocol control.

As independent providers expand, AllenHark expects to reduce its relative share of operational infrastructure.

22.22 Operational Transparency

Infrastructure operators are encouraged to publish:

- Uptime statistics.
- Incident reports.
- Planned maintenance.
- Software versions.
- Public service status.

Transparent operations improve participant confidence and encourage healthy competition between infrastructure providers.

22.23 Why Operations Matter

A decentralized protocol succeeds only if it remains reliably available.

Good operational practices ensure that:

- Users discover merchants quickly.
- Advertisements propagate efficiently.
- Trade sessions synchronize reliably.
- Infrastructure failures remain localized.
- New participants join the network easily.

Strong operations transform protocol design into dependable real-world infrastructure.

22.24 Looking Ahead

With the operational model established, the final major component of the OpenFiat ecosystem is its economic engine.

The next chapter introduces the **OPEN Token Economy**, explaining the token's purpose, utility, distribution, staking model, fee flows, treasury funding, presale structure, and the long-term economic incentives that align every participant in the OpenFiat ecosystem.